

Critical Reflection

Design and Implementation of a Real-Time Water Quality Monitoring System
Using FIWARE

Peter Jackson · 32031854
Supervisor: Dr. Carlos Eduardo da Silva
Development Project · 55-608850

Overview

Part 1: Deliverable Reflection

Requirements met, testing methodology and results, limitations

Part 2: Personal Reflection

Process, time management, what I'd do differently

Part 3: Personal Development Plan

SMART goals for future learning and development

Requirements: All 7 Specifications Met

#	Specification	Status
1	NGSI-LD data models — WFD hierarchy (WaterBody, Station, Observation)	✓ Met
2	Real-time data ingestion pipeline to time-series database	✓ Met
3	Web dashboard: station navigation, current readings, historical charts	✓ Met
4	Map-based interface with spatial navigation and polygon selection	✓ Met
5	Graphical entity management without direct API interaction	✓ Met
6	WCAG compliance with colour-blind safe colour scheme	✓ Met
7	System viability via simulation of EA historical data	✓ Met

Testing: Methodology & Rationale

Why Manual Testing?

Single-developer project with exploratory scope; automated test setup overhead not justified at this scale. Manual testing allowed faster iteration during development.

Why No Automated Tests?

Automated integration testing would be appropriate for a production deployment but introduces significant CI/CD overhead for a research prototype.

Why No Human Participants?

Target users (environmental officers, EA analysts) were not accessible in an academic context. System validated against EA open data rather than live sensor input.

Testing: NGSI-LD Pipeline Verification

[SCREENSHOT: curl command confirming entity creation in Orion-LD]

[SCREENSHOT: SQL query result from TimescaleDB confirming persistence]

Verified: entity creation → QuantumLeap subscription triggered → TimescaleDB persistence confirmed via direct SQL

Testing: Functional and Integration Tests

Google Lighthouse audits against axe-core rules mapped to WCAG 2.1 criteria (Google, n.d.)

Page	Lighthouse Score	Result
Dashboard	91 / 100	✓ Pass
Map View	96 / 100	✓ Pass
Entity Management	92 / 100	✓ Pass

[SCREENSHOT: Lighthouse result for one page]

Testing: Accessibility Audits

Google Lighthouse audits against axe-core rules mapped to WCAG 2.1 criteria (Google, n.d.)

Page	Lighthouse Score	Result
Dashboard	91 / 100	✓ Pass
Map View	96 / 100	✓ Pass
Entity Management	92 / 100	✓ Pass

[SCREENSHOT: Lighthouse result for one page]

Limitations & Future Enhancements

Simulation vs Real Sensors

Viability demonstrated via synthetic data from historical EA records. A production system would require physical sensor integration and live data ingestion.

QuantumLeap REST API Inconsistency

Inconsistent behaviour querying across multiple NGSI-LD entities required direct TimescaleDB queries via asyncpg, pragmatic but introduces tighter coupling.

Weather/Rainfall Integration (Supervisor Recommendation)

Carlos suggested integrating Open-Meteo rainfall data to contextualise readings against environmental conditions, identified as a valuable enhancement not implemented within project timeline.

No Cloud Deployment

Stack runs locally via Docker Compose. A production system would require cloud deployment for high availability and scalability.

Tools & Methods — Critical Reflection

FastAPI

Async support proved genuinely valuable: concurrent Orion-LD and TimescaleDB calls reduced latency. Flask would have required additional structuring effort for the multi-router architecture.

QuantumLeap: Encountered Limitation

QuantumLeap REST API returned inconsistent results when querying across multiple NGSI-LD entities. Rather than working around this at the API layer, historical queries were routed directly to TimescaleDB via asyncpg. Pragmatic but introduces tighter coupling. In future, contributing a fix upstream or using a more stable persistence layer from the outset would be preferable.

Git & Version Control

Git used throughout with GitHub as remote. Commits were functional but not structured, no formal branching strategy. For a larger project or team context this would be a clear gap.

Neovim + tmux

Terminal-based workflow allowed seamless context switching between editor, running Docker services, and log output in a single environment. Unconventional but effective for this stack.

Part 2: Personal Reflection

Process, planning, and what I'd do differently

Process and Time Management

Agile: What Worked

Weekly iterative sprints suited the exploratory nature of the project well. Requirements evolved significantly during development, particularly around the CSV ingest pipeline and entity hierarchy, and the sprint structure allowed these to be accommodated without derailing progress.

Trello: What Didn't Work

Task management was inconsistent. The Trello board was set up but tasks were often tracked mentally or in scattered notes rather than on the board. This made it difficult to retrospectively assess velocity or identify blockers early. In future I would enforce stricter board discipline.

Impact of Competing Priorities

Job interviews impacted the project timeline, particularly in the final weeks. This pushed report writing and testing into a compressed timeframe.

What I'd Do Differently

Start the report earlier

Report writing was left too late and compressed into the final sprint alongside testing. In future I'd write sections concurrently with development rather than after.

Structured testing from the start

Testing was done in large batches rather than continuously. The report mentions iterative sprints but testing didn't fully follow this pattern, test-driven or continuous testing would have caught issues earlier.

Placement year context

Prior exposure to industrial monitoring and SCADA systems during placement directly informed architecture decisions, particularly the value of real-time operational data and the complexity of reliable infrastructure in regulated environments.

Part 3: Personal Development Plan

SMART goals for continued learning

Personal Development Plan

Working with Docker and containerised infrastructure throughout this project has directly strengthened my understanding of cloud deployment architectures.

Goal	How	By When
Gain AWS cloud certification (Solutions Architect Associate)	Complete AWS training resources; sit the exam	6 months
Implement automated integration testing (pytest, CI/CD)	Apply to a personal or work project; complete a testing course	3 months
Deepen NGSI-LD knowledge and contribute to a data space	Follow AD4GD/WATERVERSE open repos; contribute where possible	6 months
Deploy a production-grade containerised system to cloud	Use AWS ECS/EKS on a personal or work project post-graduation	12 months

Summary

All 7 project specifications were met, with system viability demonstrated through simulation of historical EA data.

Accessibility audits scored 90+ across all pages. The NGSI-LD pipeline was verified end-to-end via curl and direct SQL validation.

Key limitations — simulation vs real sensors, QuantumLeap inconsistency, no cloud deployment — are acknowledged and inform future development priorities.

The system demonstrates the viability of FIWARE and NGSI-LD as a foundation for semantic interoperability in environmental monitoring and federated data spaces.